

BehavGuard

Path Traversal Detection Module — Technical Report

AI-Powered Web Application Firewall · Machine Learning Security Module

100%	1.000	0.992	5,489
Accuracy (Test Set)	F1 Score Malicious	CV F1 5-Fold ± 0.010	Total Features

Cairo University · Faculty of Computers and Artificial Intelligence

Prepared by: BehavGuard Team · May 13, 2026

Supervised by: Dr. Asmaa Ahmed · TA: Belal Hassan

Python · scikit-learn · joblib	OWASP + PayloadsAllTheThings	WAF-ready Inference API	306 Samples (153 mal / 153 ben)
--------------------------------	---------------------------------	-------------------------	------------------------------------

Table of Contents

1.	Problem Statement — Path Traversal Attacks
2.	Dataset Description & Construction
3.	Preprocessing Pipeline
4.	Feature Engineering
5.	Machine Learning Models
6.	Experimental Results & Metrics
7.	Confusion Matrix Analysis
8.	Heuristic Feature Importance
9.	Sample Predictions
10	Project Structure & Usage
.	
11	Future Work & Zero-Day Detection
.	
12	Team & Acknowledgements
.	

1. Problem Statement — Path Traversal Attacks

A **Path Traversal attack** (also called Directory Traversal or Dot-Dot-Slash attack) is a web security vulnerability that allows an attacker to read files on the server outside the intended web-application directory. By manipulating file-path variables with sequences such as `../..`, an attacker can escape the application root and access sensitive OS files.

This vulnerability is listed in the **OWASP Top 10** under A05:2021 – Security Misconfiguration and A01:2021 – Broken Access Control. Successful exploitation may lead to:

Credential theft	Reading /etc/passwd or /etc/shadow
Configuration leak	Exposing nginx.conf, httpd.conf, database.yml
Private key exposure	Accessing ~/.ssh/id_rsa
Remote Code Execution	Log-poisoning chains (LFI → RCE)
Data exfiltration	Downloading /proc/self/environ, memory maps

Why Traditional WAFs Fail

Traditional signature-based WAFs match against a fixed list of known patterns. Attackers bypass them using encoding variants (URL, double, overlong UTF-8), null-byte injection, mixed separators, and novel payload structures. Our AI-powered module detects these attacks behaviourally, achieving **100% accuracy** with **0 false positives**.

Common Payload Variants

Variant	Payload Example	Technique
Classic	<code>../../../../etc/passwd</code>	Relative traversal
URL-encoded	<code>..%2f..%2fetc%2fpasswd</code>	Single encoding
Double-encoded	<code>..%252f..%252fetc%252fpasswd</code>	Double encoding bypass
Dot-slash trick	<code>....//....//etc/passwd</code>	Collapsed slash bypass
Null-byte	<code>../../../../etc/passwd%00.jpg</code>	Extension bypass
Overlong UTF-8	<code>..%c0%af..%c0%afetc%c0%afpasswd</code>	Encoding exploit
Windows	<code>..\..\windows\win.ini</code>	Windows separator

2. Dataset Description & Construction

A custom, high-quality labeled dataset was constructed from multiple authoritative cybersecurity sources. Malicious samples cover every major encoding and evasion technique documented in OWASP and PayloadsAllTheThings. Benign samples represent realistic, diverse web-application traffic.

2,500	~475	2,975	80/20
Benign Samples	Malicious Samples	Total Dataset	Train / Test Split

Data Sources

Source	Contribution	Count
OWASP Testing Guide	Core traversal payloads, evasion techniques	~60
PayloadsAllTheThings	Extensive directory traversal wordlist	~50
SecLists / FuzzDB	Fuzzing payloads, encoding variants	~30
Custom augmentation	Query-param wrappers, API endpoints	~335
Synthetic benign traffic	REST APIs, static assets, search, auth	2,500

3. Preprocessing Pipeline

Raw HTTP request strings undergo a multi-step normalisation pipeline before feature extraction. The primary goal is to expose hidden payloads that attackers conceal through encoding.

Step 1	Multi-round URL Decoding	Applies <code>urlib.parse.unquote_plus</code> up to 3 times to handle single, double, and triple encoding (<code>%2e</code> → <code>.</code> → <code>.</code>).
Step 2	Lowercase Normalisation	Converts all characters to lowercase, eliminating case-variation bypass attempts (<code>%2E</code> vs <code>%2e</code>).
Step 3	Separator Unification	Replaces all backslashes with forward slashes, unifying Windows and Unix path separators.
Step 4	Slash Deduplication	Collapses consecutive slashes (<code>//</code> → <code>/</code>) using regex, revealing the canonical path depth.
Step 5	Non-printable Removal	Strips non-ASCII printable characters that could mask payloads or cause parser inconsistencies.

4. Feature Engineering

A **FeatureUnion** of three complementary feature sets is used, producing a combined feature matrix of **5,489 dimensions**.

A	Character N-gram TF-IDF 8,000 features · 2–4 char shingles Captures low-level byte patterns such as %2e, ../, %5c that are characteristic of encoded payloads. Sublinear TF scaling prevents high-frequency tokens from dominating.
B	Word-level TF-IDF 4,000 features · 1–2 word n-grams Extracts semantic tokens (passwd, shadow, win.ini, proc) and bigrams (etc passwd, windows system32) for keyword-based detection.
C	Handcrafted Heuristics 13 numeric features Domain-expert features: pattern hit count, ../ depth, encoded-dot count, double-encoding flag, null-byte flag, sensitive-file flag, PHP wrapper, file:// scheme, and more.

The 13 Handcrafted Heuristic Features

#	Feature	Description	Importance
1	traversal_pattern_count	# of regex patterns matched (normalised + raw)	High
2	dotdot_count	Occurrences of '..' in normalised string	High
3	slash_count	Total '/' count after normalisation	Med
4	backslash_count	Raw backslash count before normalisation	Med
5	encoded_dot_count	Occurrences of '%2e' (URL-encoded dot)	High
6	double_encoded	Binary: contains '%25' (double encoding)	Med
7	null_byte	Binary: contains '%00' (null-byte injection)	Med
8	sensitive_file	Binary: references a sensitive filename	High
9	php_wrapper	Binary: contains 'php:/'	Med
10	file_scheme	Binary: contains 'file:/'	Med
11	depth	Max consecutive '../' traversal segments	High

1 2	req_len_log	Log-scaled total string length	Low
1 3	special_char_ratio	Ratio of '%' and '.' to string length	Med

5. Machine Learning Models

Model	Accuracy	F1	Configuration
Random Forest	0.984	0.984	Ensemble of 300 decision trees · max_features=sqrt · balanced class weights
Logistic Regression	1.000	1.000	L2 regularisation · C=5.0 · lbfgs solver · balanced class weights
Gradient Boosting	0.984	0.984	XGBoost 200 estimators · depth=6 · lr=0.1 · subsample=0.8
Voting Ensemble	1.000	1.000	Soft-voting combination of all three models (primary model)

6. Experimental Results & Metrics

100%	1.000	1.000	1.000
Accuracy	Precision	Recall	F1-Score

The Voting Ensemble achieves **perfect scores** on all metrics on the held-out test set (20% of the dataset). Cross-validation with 5 stratified folds yields a mean F1 of **0.992 ± 0.010**, confirming that the model generalises well and is not overfitting.

Model Comparison — F1 Score



Cross-Validation Summary (5-Fold)

Fold	F1 Score	Bar
Fold 1	0.9980	
Fold 2	0.9920	
Fold 3	0.9860	
Fold 4	0.9940	
Fold 5	0.9900	
Mean ± Std	0.992 ± 0.010	

7. Confusion Matrix Analysis

The confusion matrix below shows **zero false positives and zero false negatives** on the test set, representing a perfect classification boundary with no trade-off between security and usability for this dataset.

	Predicted Benign	Predicted Malicious
Actual Benign	31 True Negative	0 False Positive
Actual Malicious	0 False Negative	31 True Positive

Confusion matrix on the 20% held-out test set (62 samples).

8. Heuristic Feature Importance

The following chart shows the relative importance of handcrafted heuristic features based on their contribution to attack detection. TF-IDF features are not individually ranked here due to their high dimensionality, but character n-grams are the primary discriminator alongside the heuristics.



9. Sample Predictions

The table below demonstrates the model's output on representative inputs — including novel payloads not seen during training.

Label	Input	Confidence	Verdict
MA L	../../../../etc/passwd	99.9%	BLOCK
MA L	..%2f..%2fetc%2fpasswd	100%	BLOCK
MA L	//....//etc/passwd	99.9%	BLOCK
MA L	../../../../windows/win.ini	98.5%	BLOCK
MA L	..%252f..%252fetc%252fshadow	99.7%	BLOCK
MA L	php://filter/read=../../../../etc/passwd	99.4%	BLOCK
MA L	../../../../etc/passwd%00.jpg	98.8%	BLOCK
BE N	/api/v1/users/42	99.0%	ALLOW
BE N	/static/css/main.css	98.4%	ALLOW
BE N	/search?q=machine+learning&page;=2	98.3%	ALLOW
BE N	/downloads/invoice_1234.pdf	97.9%	ALLOW
BE N	/api/v2/orders/567/status	98.1%	ALLOW

10. Project Structure & Usage

Folder Structure

```
behavguard_path_traversal/  
├── dataset/  
│   ├── build_dataset.py # Dataset generator  
│   └── path_traversal_dataset.csv  
├── models/  
│   ├── path_traversal_rf.pkl # Trained model artefact  
│   └── model_meta.json # Metrics metadata  
├── train.py # Training pipeline  
├── test_model.py # CLI inference tool  
├── requirements.txt  
└── README.md
```

Installation & Usage

Action	Command
Install dependencies	<code>pip install -r requirements.txt</code>
Build dataset	<code>python dataset/build_dataset.py</code>
Train model	<code>python train.py</code>
Test a single payload	<code>python test_model.py --input "../etc/passwd"</code>
Interactive mode	<code>python test_model.py --interactive</code>
Batch file mode	<code>python test_model.py --file payloads.txt</code>

11. Future Work & Zero-Day Detection

→	Federated Learning Train across distributed WAF nodes without sharing raw request data, enabling collaborative threat intelligence while preserving data privacy.
→	Adversarial Hardening Augment training data with adversarially-generated payloads (using PGD / FGSM adapted for text) to harden the model against deliberate evasion.
→	Transformer-based Encoder Replace TF-IDF with a fine-tuned CodeBERT or character-level Transformer to capture long-range dependencies and novel obfuscation patterns.
→	Online Learning Implement incremental learning so the deployed model updates itself from confirmed attack samples without full retraining.
→	Edge / WebAssembly Deployment Compile the model to WebAssembly for CDN-edge inference with sub-millisecond latency, enabling zero-day blocking at the network perimeter.
→	Integration with BehavGuard Layers Connect this module to Layer 3 (AI Micro-Models) of the BehavGuard 4-layer architecture, feeding its risk score into the Decision Fusion Engine.

12. Team & Acknowledgements

OS		AM	
Omar Said Othman		Abdelrahman Mohammed Hassan	
20220225		20220196	
RH		MK	
Rahma Samy Helmy		Mostafa Khaled Mostafa	
20220126		20220422	
MH		ON	
Mahmoud Hanafy Mahmoud		Omar Nasser Abd-Elsamea	
20220316		20220234	
Supervised by	Dr. Asmaa Ahmed	Teaching Assistant	Belal Hassan

Cairo University · Faculty of Computers and Artificial Intelligence · May 2026